
Casos de Teste com JUnit

Da especificação à execução automatizada no VS Code



Prof. Dr. João Choma

Teste e Qualidade de Software

github.com/JoaoChoma

Semana 02

Do caso ao código



Um caso manual descreve entradas, ações e resultado esperado.

Dado: compra comum de R\$ 200,00

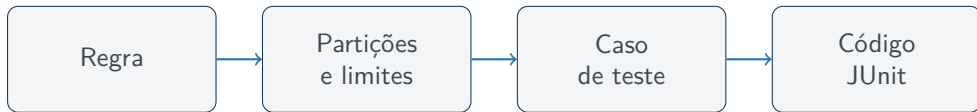
Quando: o frete é calculado

Então: o frete deve ser R\$ 0,00

No JUnit, a mesma intenção se torna código repetível e verificável.



- Explicar o papel do JUnit no teste unitário.
- Organizar preparação, ação e verificação.
- Criar e executar testes no VS Code.
- Usar asserções, exceções e parâmetros.
- Selecionar positivos, negativos e limites.
- Interpretar uma falha sem esconder o defeito.



O teste automatizado não substitui o projeto do caso: ele o executa.



- Identifica e agrupa métodos de teste.
- Fornece asserções para comparar resultados.
- Integra-se a IDEs e ferramentas de build.
- Produz resultados reproduzíveis.
- Permite repetir a suíte após cada alteração.

Versão usada

JUnit Jupiter, a API moderna do JUnit 5.



Boa candidata	Evite no primeiro exemplo
regra de cálculo	banco real
validação de entrada	API externa
transformação de dados	interface completa
decisão de negócio	relógio ou rede sem controle

Testes pequenos são rápidos e localizam melhor a causa da falha.



```
@Test
void deveSomarDoisValores() {
    // Arrange
    Calculadora calculadora = new Calculadora();
    // Act
    int resultado = calculadora.somar(2, 3);
    // Assert
    assertEquals(5, resultado);
}
```

Cada método deve comunicar uma regra de comportamento.



Anotação	Uso
@Test	identifica um teste
@BeforeEach	prepara antes de cada teste
@AfterEach	libera recursos
@DisplayName	mostra nome legível
@ParameterizedTest	repete com vários dados
@CsvSource	fornece linhas de parâmetros



```
assertEquals(0.0, frete, 0.001);
assertTrue(resultado.isValid());
assertFalse(lista.isEmpty());
assertNull(usuario.getBloqueio());
assertNotNull(pedido.getId());
assertThrows(IllegalArgumentException.class,
    () -> servico.calcular(-1, false));
```

Para ponto flutuante, informe uma tolerância.



Prefira

```
void deveIsentarFreteQuandoCompraComumAtingeDuzentos()
```

Evite

```
void teste1()  
void testarFrete()
```

O nome ajuda a interpretar a falha antes de abrir o método.

JUnit no VS Code



```
exemplo-junit/  
|-- pom.xml  
|-- .vscode/extensions.json  
`-- src/  
    |-- main/java/br/uem/testes/  
    |   |-- CalculadoraFrete.java  
    |   `-- ValidadorSenha.java  
    `-- test/java/br/uem/testes/  
        |-- CalculadoraFreteTest.java  
        `-- ValidadorSenhaTest.java
```



- 1 Instale JDK 17 ou mais recente.
- 2 Instale **Extension Pack for Java**.
- 3 Abra a pasta exemplo-junit.
- 4 Aguarde a importação do pom.xml.
- 5 Abra a visão **Testing** pelo frasco.

O arquivo `extensions.json` recomenda as extensões.



- **Run Test:** executa o método selecionado.
- **Debug Test:** permite usar breakpoints.
- **Testing:** organiza classes, resultados e mensagens.
- `mvn test`: alternativa pelo terminal com Maven.

Cores são um resumo

Verde: as expectativas foram atendidas. Vermelho: houve divergência ou erro.



```
expected: <0.0> but was: <20.0>
```

- 1 A regra e o esperado estão corretos?
- 2 Os dados ativam a condição pretendida?
- 3 O defeito está na produção ou no teste?
- 4 Existe dependência de estado ou ambiente?

Não altere o esperado apenas para obter verde.

Exemplo guiado



- RF-01: compra comum a partir de R\$ 200 tem frete grátis.
- RF-02: premium tem frete grátis a partir de R\$ 100.
- RF-03: nos demais casos, o frete custa R\$ 20.
- RF-04: valor negativo deve ser rejeitado.

O método recebe valor e perfil e devolve o valor do frete.



Condição	Entrada	Esperado
comum abaixo	199,99	20,00
comum no limite	200,00	0,00
comum acima	200,01	0,00
premium abaixo	99,99	20,00
premium no limite	100,00	0,00
premium acima	100,01	0,00
entrada inválida	-0,01	exceção



```
public double calcular(double valor, boolean premium) {  
    if (valor < 0) {  
        throw new IllegalArgumentException(  
            "valorCompra nao pode ser negativo");  
    }  
    if (valor >= 200.0 ||  
        (premium && valor >= 100.0)) {  
        return 0.0;  
    }  
    return 20.0;  
}
```

O primeiro teste verifica a fronteira comum



```
@Test
@DisplayName("Compra comum no limite: frete gratis")
void deveIsentarFreteQuandoAtingeDuzentos() {
    CalculadoraFrete calc = new CalculadoraFrete();

    double frete = calc.calcular(200.0, false);

    assertEquals(0.0, frete, 0.001);
}
```



```
@Test
void deveRejeitarValorNegativo() {
    IllegalArgumentException erro = assertThrows(
        IllegalArgumentException.class,
        () -> calculadora.calcular(-0.01, false));

    assertEquals(
        "valorCompra nao pode ser negativo",
        erro.getMessage());
}
```



```
@ParameterizedTest
@CsvSource({
    "199.99, false, 20.0",
    "200.00, false, 0.0",
    " 99.99, true, 20.0",
    "100.00, true, 0.0"
})
void calculaNosLimites(double valor,
    boolean premium, double esperado) {
    assertEquals(esperado,
        calculadora.calcular(valor, premium), 0.001);
}
```



```
class CalculadoraFreteTest {  
    private CalculadoraFrete calculadora;  
  
    @BeforeEach  
    void preparar() {  
        calculadora = new CalculadoraFrete();  
    }  
  
    @Test  
    void deveCobrarAbaixoDoLimite() {  
        assertEquals(20.0,  
            calculadora.calcular(199.99, false), 0.001);  
    }  
}
```




Comece em sala com:

```
if (valor > 200.0 ||  
    (premium && valor > 100.0)) {  
    return 0.0;  
}
```

Os casos 200,00 e 100,00 falham. A diferença entre $>$ e $\geq$ explica a divergência.



- ① Ler regras e destacar condições.
- ② Criar partições e limites.
- ③ Escrever um teste que falha.
- ④ Executar e interpretar a mensagem.
- ⑤ Corrigir a fronteira.
- ⑥ Executar toda a suíte.
- ⑦ Refatorar com a proteção dos testes.



- Um comportamento principal por método.
- Independência e determinismo.
- Nomes que revelam condição e resultado.
- Sem rede, banco ou relógio real na unidade.
- Positivos, negativos e limites.
- Suíte executada após mudanças relevantes.



Antipadrão	Consequência
teste1	falha pouco informativa
várias regras no método	causa pouco localizada
depende do teste anterior	ordem altera resultado
só caminho feliz	bordas sem cobertura
alterar o esperado	defeito escondido

Atividade dos alunos



Implemente `ValidadorSenha.ehValida(String)`.

- RS-01: senha obrigatória.
- RS-02: 8 a 20 caracteres, inclusive.
- RS-03: ao menos uma maiúscula.
- RS-04: ao menos um algarismo.
- RS-05: nenhum espaço.

O projeto contém um esqueleto para começar.



- ① Tabela de partições válidas e inválidas.
- ② Limites 7, 8, 9, 19, 20 e 21.
- ③ Pelo menos oito testes JUnit.
- ④ Um teste parametrizado.
- ⑤ Implementação com a suíte passando.
- ⑥ Justificativa dos casos escolhidos.



Entrada	Regra	Esperado
null	obrigatoriedade	falso
Abc1234	7 caracteres	falso
Abc12345	mínimo 8	verdadeiro
20 caracteres válidos	máximo 20	verdadeiro
21 caracteres válidos	acima do máximo	falso
abcdefgh1	sem maiúscula	falso
Abcdefghi	sem número	falso
Abcd 123	espaço	falso

Próxima etapa



O JUnit produz evidência, mas ainda precisamos definir:

- escopo e riscos prioritários;
- cobertura dos requisitos;
- níveis e tipos de teste;
- resultados que sustentam a liberação.

Próximos artefatos

Plano de teste, casos de teste e matriz de rastreabilidade.



- Salas possuem capacidades e recursos diferentes.
- Reservas têm data, horário, turma e responsável.
- Conflitos, manutenção e horário devem ser respeitados.
- Operações dependem do perfil do usuário.
- Alterações afetam notificações e histórico.

O domínio possui relações e riscos suficientes para um plano.



ID	Requisito
RF-01	reservar sala disponível para turma compatível
RF-02	impedir sobreposição na mesma sala
RF-03	impedir turma maior que a capacidade
RF-04	bloquear sala em manutenção
RF-05	permitir reservas entre 07h30 e 22h30
RF-06	só coordenação altera reserva de outro professor
RF-07	cancelamento libera horário e registra histórico
RF-08	alteração ou cancelamento gera notificação



- RNF-01: busca responde em até 2 segundos.
- RNF-02: operações possuem trilha de auditoria.
- RNF-03: acesso limitado às unidades autorizadas.

Riscos críticos

Dupla ocupação, capacidade insegura, alteração sem autorização e falha de notificação.



- Objetivo, escopo e exclusões.
- Riscos e prioridades.
- Estratégia por nível e tipo.
- Ambiente, dados, perfis e ferramentas.
- Responsáveis e cronograma.
- Critérios de entrada, saída e suspensão.



Cada caso deverá conter:

- ID, título, requisito e risco;
- prioridade e nível;
- pré-condições e dados;
- passos e resultado esperado;
- resultado real, status e evidência.

Inclua positivos, negativos, limites, permissões, conflitos e estados.



Req.	Risco	Casos	Nível
RF-02	R-01	CT-RES-004, 005	integração/sistema
RF-03	R-02	CT-RES-006 a 008	unidade/sistema
RF-06	R-03	CT-RES-012, 013	sistema
RF-08	R-04	CT-RES-016 a 018	integração

Requisitos sem teste e testes sem propósito ficam evidentes.



- 1 Esclarecer requisitos ambíguos.
- 2 Identificar e priorizar riscos.
- 3 Delimitar plano e estratégia.
- 4 Projetar casos e dados.
- 5 Construir a matriz.
- 6 Revisar cobertura e consistência.
- 7 Selecionar regras para JUnit.